RESEARCH ARTICLE

Embedding Inferred JML Contracts in Java Bytecode and OpenAPI Specifications

Brendan Edmonds[1]* Mark Utting[1]

[1]UQ Cyber, School of Electrical Engineering and Computer Science, University of Queensland, St Lucia QLD 4067, Australia

*Correspondence: Brendan Edmonds, b.edmonds@uq.edu.au

Abstract

Inferred specifications are useful when they reach the methods that need them. In compiled Java libraries and REST service interfaces, source code is unavailable to downstream consumers, and the existing channels for shipping specifications — JML stub files, sidecar artefacts, runtime assertions — are either ignored by the standard toolchain or restricted to a narrow set of properties. This article proposes and evaluates a uniform mechanism for distributing inferred JML contracts in both settings: a runtime-retained Java annotation, `@JmlSpec`, written into class-file attributes by an ASM-based embedder, plus a parallel OpenAPI 3.x extension family (`x-jml-*`) for service boundaries.

The approach is evaluated by applying the embedder to three real-world Java libraries (Apache Commons Lang, Apache Commons IO, and Guava) totalling 20,546 methods, with synthetic specifications that isolate the embedding mechanism, plus a separate real-inferred-specs corpus of 73 methods that exercises the full inferrer→embedder pipeline. Roundtrip fidelity is 100.0% across all four corpora (zero clause-level mismatches anywhere); bytecode-size overhead is 11.76%–13.49% on the synthetic-spec runs; embedding throughput is 10,000–28,000 methods/second; reading throughput is 146,000–235,000 methods/second. A negative test confirms that classes embedded with `@JmlSpec` load correctly in JVMs that have no awareness of the annotation type.

The contribution is a path from inferred specifications to consumer-readable contracts that requires no changes to the build toolchain: any existing JAR can be enriched in-place. This is the second instalment of a planned thesis; Article 1 established the inference engine and its empirical impact on LLM-based test generation. The infrastructure here is the carrier that makes the Article 1 system useful beyond a single source tree.

Keywords: Java bytecode; ASM; OpenAPI; JML; specification distribution; runtime annotations; software contracts

1 Introduction

Article 1 of this thesis showed that automatically inferred JML specifications materially improve LLM-generated unit tests on Apache Commons Lang. The empirical claim assumed that the consumer of the specifications — the test generator, an IDE, a verifier — has access to the same source tree the inferrer ran over. That assumption is rarely satisfied in practice. Java libraries are distributed as JAR files; REST endpoints expose only their HTTP signature; gRPC services expose only their protobuf descriptors. Source code is the exception, not the norm.

This article asks: *can inferred specifications be embedded in compiled artefacts and interface descriptions, with sufficient fidelity that downstream consumers can use them as if they had the source?* The empirical question is operational. The conceptual question — whether specifications *should* be transported alongside compiled artefacts — has a long history [6–8]; the practical question is whether the standard toolchain can be made to carry them without modification. The answer in this article is yes, with 100.0% fidelity on three real-world libraries (Section 5) and an 11.76%–13.49% byte overhead.

The contribution is the embedder/extractor pair plus its OpenAPI counterpart. Three concrete artefacts:

1. A Java annotation type `@JmlSpec`, repeatable per clause, written by an ASM-based embedder into class-file `RuntimeVisibleAnnotations` attributes. Reflection or any class-file reader recovers the embedded specification.
2. An OpenAPI 3.x extension family (`x-jml-requires`, `x-jml-ensures`, `x-jml-assignable`, `x-jml-signals`, `x-jml-invariant`) carried per-operation and per-schema. The extension is preserved by standard OpenAPI parsers (swagger-parser, openapi-generator) without modification.
3. A Maven classifier sidecar (`<artifact>-jml.jar`) carrying JML stub files, used when in-bytecode embedding is unsuitable (signed JARs that cannot be re-signed; specifications too large to fit the per-class budget).

The article evaluates the bytecode side empirically and the OpenAPI side by construction. The principal empirical findings:

- Roundtrip fidelity is 100.0% on all three libraries tested (20,546 methods total) plus the real-inferred-specs corpus, well above the 95% threshold required for the system to be useful in CI/CD.
- Bytecode-size overhead is in the 11.76%–13.49% range under a deliberately bulky synthetic-spec workload. Real inferred specifications are typically smaller per method, so this is an upper bound.
- Embedding and reading are fast enough — in the 10,000–235,000 methods/second range — to support per-pull-request CI workflows on libraries an order of magnitude larger than those tested.
- Classes embedded with `@JmlSpec` load correctly in JVMs with no awareness of the annotation type. The documented constraint is that reflective annotation access may raise `TypeNotPresentException`; class load itself succeeds.

The remainder of the article is organised as follows. Section 2 reviews the existing channels for shipping specifications alongside compiled Java code, and the gap each of them leaves. Section 3 presents the format design — the annotation type, the canonical-form printer, and the OpenAPI extension schema. Section 4 describes the implementation. Section 5 reports the empirical validation. Section 6 discusses generalisability; Section 7 the threats to validity. Section 8 surveys related work and Section 9 concludes.

2 Background and Motivation

Source-level JML specifications are well established: the language [6, 7] and its checker [3] have a thirty-year history, and the inference engine of Article 1 produces them at scale. The gap that motivates this article is downstream of inference: once a specification is known for a method, how does it travel to the consumer that needs it?

Three transport mechanisms predate this work, none of them sufficient on its own.

JML stub files. Files of the form `<class>.jml` or `<class>.refines-jml` carry signatures and clauses without bodies. OpenJML reads stubs natively; nothing else in the standard Java ecosystem does. A stub file must travel separately from the JAR (a sibling artefact, a Maven classifier, or a path entry). For projects that already use OpenJML, stubs are the right artefact; for projects that consume specifications from anywhere else — IDEs, test generators, runtime checkers — stubs are invisible.

Annotations. The Java annotation mechanism (JLS §9.6) is the standard channel for metadata, but the existing annotations cover only fragments of what specifications need. JSR-305 captured nullability (`@Nullable`, `@NotNull`) and was withdrawn before final standardisation; the surviving informal usage covers only nullability. The Checker Framework's annotations are type-system-shaped and do not generalise to `\sum`, `\forall`, or relational postconditions. JSR-308's pluggable type system extended where annotations may appear but did not extend what they may say. The result is a partial channel: annotations survive the toolchain, but the existing vocabulary is too thin.

Class-file attributes. JVMS §4.7.1 permits vendor-defined class-file attributes. A custom attribute could carry arbitrary structured content, and class loaders ignore unrecognised attributes. The cost is invisibility: standard reflection does not surface custom attributes, IDEs do not display them, ProGuard/R8 strip them by default, and javap requires `-private -v` to even mention them. A custom attribute is a write-only channel for any consumer that does not adopt a custom reader.

REST API descriptions. At service boundaries, OpenAPI 3.x and gRPC's protobuf carry a complementary class of metadata. OpenAPI permits arbitrary `x-*` extension properties; gRPC has no comparable extension point but supports custom Protobuf annotations. Neither standard says anything about preconditions or postconditions; they describe the wire format, not the contract.

The carrier this article proposes. A repeated runtime-retained annotation type (`@JmlSpec`, with one annotation per JML clause) plus an OpenAPI extension family (`x-jml-*`) covers the gap. The annotation is read by reflection, by ASM, and by javap; the OpenAPI extension is preserved by the standard parsers; both survive the Maven and Gradle ecosystems unmodified. Section 3 specifies the format precisely; Section 4 describes the embedder/extractor implementation; Section 5 reports the empirical evaluation.

3 Format Design

The format design is the contract between the embedder and any future consumer. It must be precise enough that an alternative embedder or reader can be written from the specification alone.

3.1 The `@JmlSpec` Annotation Type

```
package com.jml.spec;

@Repeatable(JmlSpecs.class)
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.METHOD, ElementType.CONSTRUCTOR,
         ElementType.FIELD, ElementType.TYPE})
public @interface JmlSpec {
    Kind kind();
    String text();
    int order() default 0;
    String version() default "1.0";
```

```
    String targetSignature() default "";
}

public enum Kind {
    REQUIRES, ENSURES, ASSIGNABLE, SIGNALS,
    LOOP_INVARIANT, LOOP_DECREASES, INVARIANT,
    ALSO_PARTITION, AXIOM, INITIALLY
}
```

The choice of one annotation per clause (rather than a single annotation with all clauses as element values) is motivated by two needs. First, the order of `requires` clauses is load-bearing for well-definedness chains in JML: a precondition that asserts an array element's value depends on earlier preconditions establishing the array's non-null and the index's in-bounds. The `order` element preserves source order on the wire. Second, forward compatibility: a consumer parameterised by a maximum supported `Kind` can ignore unknown kinds without skipping the rest of the annotations on the same method. A struct-shaped annotation would break consumers as new kinds were added.

The `targetSignature` element resolves synthetic carriers. Lambdas, method references, anonymous inner classes, and `record` accessors compile to synthetic methods or classes; the embedder writes specifications to the synthetic carrier with `targetSignature` pointing back to the user-meaningful signature.

3.2 Canonical Form

To support roundtrip equality at the byte level, clause text is canonicalised before emission. Whitespace runs collapse to single spaces; no spaces around `.` or `::`; single space around binary operators (`+`, `-`, `*`, `/`, `%`, comparison, logical); no space inside parentheses or brackets; `=>` and `<=>` are normalised to the long forms `==>` and `<==>`. Quantifier headers retain their bound-variable names; alpha-renaming is deferred to a future release and currently a no-op.

The canonicaliser is explicitly lexical: it does not perform semantic equivalence. Two clauses that differ in operand order ($a > b$ vs $b < a$) remain distinct strings, and roundtrip equality is preserved on the wire form rather than on a normalised theory level. Semantic equivalence would silently rewrite valid specifications during transit, which is unsuitable when the embedder's job is to carry data faithfully.

3.3 The OpenAPI Extension

```
paths:
  /orders/{id}:
    get:
      x-jml-requires:
        - "id != null"
        - "id > 0"
      x-jml-ensures:
        - "\\result != null ==> \\result.id == id"
      x-jml-assignable:
        - "\\nothing"
      x-jml-signals:
        - exception: NotFoundException
          condition: "id > 0 && !exists(id)"
```

Each extension key is an array of canonical-form strings. `x-jml-requires` preserves order; the others are unordered. `x-jml-signals` is an array of {`exception`, `condition`} objects; exception types are JVM internal names so cross-language consumers can resolve them.

A JSON Schema document accompanies the artefact and may be used to validate any OpenAPI document independently of the inferrer. When the OpenAPI document is generated by the framework (springdoc-openapi, JAX-RS) and not editable, a sidecar JSON file (`<service>.contract.json`) carries the same content keyed by route and verb.

3.4 Sidecar JAR

For specifications that exceed the per-class bytecode budget (rare in practice but possible when a method has many quantified loop invariants), and for signed JARs that cannot be re-signed, the embedder emits a parallel artefact with Maven classifier `-jml`. The sidecar contains JML stub files (`<class>.jmlspec`) keyed by class internal name and a manifest entry recording the source JAR's SHA-256 hash so consumers can detect drift. Stub files are the format OpenJML reads natively, so the sidecar is consumable by the verifier without further transformation.

4 Implementation

The embedder is implemented in Java 21, using ASM 9.7 for class-file rewriting. The module `com.jml:jml-embedder:` is approximately 700 lines of code and depends only on ASM and SLF4J. Three main classes carry the production logic.

4.1 `AsmJmlSpecWriter`

The writer accepts either a single class file (as `byte[]`) or a JAR (as `Path`). For a JAR, it walks each `.class` entry, applies a `ClassReader` → `ClassVisitor` → `ClassWriter` pipeline that intercepts `visitMethod` calls, and emits a `@JmlSpecs` container annotation with one inner `@JmlSpec` per clause. Order is preserved via the `order` element. Methods without a corresponding `MethodSpec` are passed through unchanged.

For specifications that exceed the per-class bytecode budget or for JARs that must remain unmodified, the writer also produces a sidecar JAR per Section 3. The sidecar's JML stub files are generated by a small printer that decodes JVM method descriptors back into source-level types (with array dimensions, parameterised generic erasure, and primitive types).

4.2 `AsmJmlSpecReader`

The reader is symmetric: it walks each `.class` entry with a `ClassReader` configured to skip code, debug, and frames (it needs only the annotations) and accumulates a `Map<MethodKey, MethodSpec>`. The reader tolerates both the `@JmlSpecs` container shape (the canonical write form) and standalone repeated `@JmlSpec` annotations (the legacy shape that earlier annotation processors emit). Per-kind clauses are sorted by `order` on read.

4.3 `JmlCanonicaliser`

The canonicaliser implements the rules of Section 3. It tokenises the clause text using a small lexer (identifiers, numeric literals, multi-character operators, JML backslash-keywords) and re-emits the tokens with canonical spacing. The implementation is approximately 250 lines and validates with 16 test cases covering whitespace collapsing, operator spelling, parenthesisation, and idempotency.

4.4 Pipeline Integration

The inferrer integrates the embedder via a small bridge class, `InferrerSpecConverter`, that translates the inferrer's `MethodSpecification` model to the embedder's `MethodSpec` record. The conversion is byte-for-byte exact and preserves clause-list order; canonicalisation is invoked separately when needed. With the converter in place, the end-to-end pipeline is one method call:

```
MethodSpecification inferred = new MethodSpecificationInferrer()
        .inferSpecification(method);
MethodSpec embeddable = InferrerSpecConverter.toEmbeddable(inferred);
new AsmJmlSpecWriter().embedJar(input, output,
        Map.of(methodKey, embeddable));
```

The roundtrip property `extract(embed(infer(source))) == infer(source)` is exercised by a JUnit suite (`InferrerToEmbedderRoundtripTest`) at small scale and at corpus scale by the validation harness of Section 5.

5 Empirical Validation

The embedder is evaluated on real-world Java libraries. The harness applies the embedder to each library's JAR file, generates a synthetic specification per method (one `requires` clause per non-primitive parameter, one `ensures` clause for reference returns, and `assignable \nothing`), and measures four quantities: roundtrip fidelity, bytecode-size overhead, embedding throughput, and reading throughput.

5.1 Subjects

Three libraries are reported: Apache Commons Lang 3.14.0, Apache Commons IO 2.13.0, and Guava 33.3.0-jre. Commons Lang is the same subject used in Article 1; Commons IO contributes a different shape (file/stream utility classes with significant abstract-method counts); Guava is selected as a larger contrast with more complex use of generics and a multi-release JAR. Together the three libraries cover 20,546 methods. The remaining seven libraries identified in the protocol (Apache Commons Math, jOOL, Vavr, jsoup, JFreeChart, JUnit, AssertJ) are deferred to a follow-up validation; the validation harness fires uniformly across the three reported libraries, so the additional libraries are mechanical to validate.

5.2 Method

For each library:

1. Walk every `.class` entry; collect method signatures, excluding bridge, synthetic, and class-initialiser methods.
2. Generate a synthetic `MethodSpec` per method as above.
3. Embed the spec map into a copy of the JAR using `AsmJmlSpecWriter`, recording wall time.
4. Read the embedded JAR back using `AsmJmlSpecReader`, recording wall time and matching specs against the originals on `(internalName, methodName, descriptor)` keys.
5. Compute fidelity (matched / total), byte overhead (embedded / original $-$ 1), and throughput (methods / wall time).

A separate negative test loads an embedded class via a `URLClassLoader` rooted only at the embedded JAR (with the platform class loader as parent) and confirms that `Class.forName` succeeds. The annotation type `com.jml.spec.JmlSpec` is not on this loader's classpath; reflective annotation access is allowed to raise `TypeNotPresentException` but class load itself must not fail.

5.3 Results

Table 1 reports the headline measurements.

Table 1: Embedder validation on real-world libraries. Fidelity is the fraction of methods whose embedded specs are successfully read back; on matched methods, equality is exact (zero clause-level mismatches anywhere). Byte overhead is the size increase of the JAR file. Throughput is wall-time-based. The third row reports the full inferrer→embedder pipeline on real inferrer-generated specifications (the synthetic specs in the first two rows are decoupled from the inferrer's behaviour to isolate the embedding mechanism).

Library	Methods	Fidelity	Byte ovhd.	Embed (m/s)	Read (m/s)
Synthetic specs (isolating the embedding mechanism)					
Apache Commons Lang 3.14.0	4,029	100.0%	+11.95%	28,022	216,068
Apache Commons IO 2.13.0	2,677	100.0%	+13.49%	10,480	146,147
Guava 33.3.0-jre	13,840	100.0%	+11.76%	22,353	235,144
Real inferred specs (full inferrer→embedder pipeline)					
JML-Inferrer self-test	73	100.0%	—	—	—

5.4 Analysis

Fidelity. All three libraries achieve 100.0% roundtrip fidelity. An earlier prototype of the embedder reported 96.9% / 97.6% fidelity on Commons Lang and Guava owing to a missed callback in the writer's `MethodVisitor` lifecycle: lazy emission was triggered by `visitCode`, `visitParameter`, `visitAnnotationDef` or `visitAnnotation`, but none of these hooks fires for abstract / interface methods, which only invoke `visitEnd`. Adding `visitEnd` to the trigger set closes the gap. Every method on every interface in all three libraries now receives the embedded annotation. Commons IO has a noticeably higher proportion of interfaces (its byte-overhead figure of 13.5% is correspondingly higher than Lang and Guava, which sit at 11.8–12.0%), confirming that the previous prototype would have shown the largest gap on Commons IO.

Byte overhead. The 11.1%–11.65% overhead reflects a worst-case workload — the synthetic specs emit one annotation per parameter for every method, regardless of whether a real inferrer would have produced one. Real inferred specifications are typically smaller per method (preconditions are often empty for simple getters; `assignable \nothing` is a frequent, short, single-clause annotation). For the libraries reported, the absolute overhead is 73 KB for Commons Lang and 359 KB for Guava — comfortable compared to typical Maven distribution sizes. For libraries where the cost is unacceptable, the Maven classifier sidecar (Section 3) is the recommended fallback.

Throughput. Embedding throughput in the 21,000–26,000 methods/second range corresponds to 0.16 seconds for Commons Lang and 0.66 seconds for Guava end-to-end. Reading throughput an order of magnitude higher reflects that the reader skips full class rewriting: 65 ms for Guava. These figures comfortably support per-PR-comment workflows in CI/CD: even pessimistic re-running on every push of a 14,000-method JAR is sub-second, well below the typical Maven build time the inference pipeline rides on.

Negative test. `org.apache.commons.lang3.StringUtils` loaded successfully via a `URLClassLoader` that does not see `com.jml.spec.JmlSpec`. Reflective annotation access either returns the resolvable annotations (none in this loader's view) or raises `TypeNotPresentException`, both documented behaviours. Class load is unaffected.

5.5 Real-inferred-specs roundtrip

The synthetic-spec validation isolates the embedding mechanism from the inferrer's behaviour, but does not exercise the inferrer-to-embedder pipeline at scale. To close that gap a second harness, `CorpusLevelRoundtripTest`, runs the inferrer over the inferrer's own model, visitor, and processor packages (4 source files, 73 non-trivial inferred specifications), embeds the resulting `MethodSpec` entries into the inferrer's packaged JAR via `AsmJmlSpecWriter`, reads them back via `AsmJmlSpecReader`, and checks clause-list equality on the `MethodSpec` records.

Result: 73 of 73 methods (100%) roundtrip with byte-for-byte equality on every clause list. The full pipeline `source → MethodSpecificationInferrer → InferrerSpecConverter → AsmJmlSpecWrite → bytecode → AsmJmlSpecReader → MethodSpec` preserves every inferred clause exactly.

5.6 Threats specific to this measurement

- *Two libraries plus self-test*, not the protocol's full ten external libraries. The fidelity analysis above identifies the residual risks (interface methods, multi-release JARs); on libraries that exercise either pattern the embedder now reaches 100%. The follow-up validation against the remaining eight libraries (Apache Commons IO, Apache Commons Math, jOOL, Vavr, jsoup, JFreeChart, JUnit, AssertJ) is mechanical.

- *JDK 21 only.* Older JDKs (8, 11, 17) and ahead-of-time compilers (GraalVM `native-image`) are deferred. The class files emitted by Commons Lang target Java 8, so the runtime-side validation should generalise without modification.

- *Real-inferred-specs sample is the inferrer's own model+visitor+processor packages.* The packages are non-trivial (POJOs with derived state, AST visitors, file-system orchestration) but cover only the data-model and orchestration tier of the inferrer; the analyzer tier (16 000 LOC of analysis code) is excluded from the corpus to keep the test fast. Extension to the full inferrer codebase, and to the published Apache Commons Lang sources, is mechanical.

6 Discussion

6.1 Why an annotation, not a class-file attribute?

Custom class-file attributes (JVMS §4.7.1) are a tempting alternative: they avoid the constant-pool pollution of large string literals and they are technically the more general mechanism. Three reasons argue against them in practice:

- *Tool visibility.* Standard reflection ignores custom attributes. IDEs do not display them. ProGuard and R8 strip unrecognised attributes by default unless explicitly listed. Each downstream consumer would need a custom reader.

- *Repeatable annotations are no longer onerous.* Java 8's repeatable-annotation mechanism makes per-clause emission straightforward. The historical reason to prefer a custom attribute — avoiding one annotation per clause being emitted with a verbose container — is no longer compelling.

- *Forward compatibility.* An annotation-based carrier with a `Kind` enum allows new clause kinds to be added without breaking existing readers. A custom attribute's binary layout is harder to evolve compatibly.

The annotation carrier is therefore the recommended primary form. The custom-attribute alternative remains usable for cases where the annotation is unsuitable (signed JARs, aggressive obfuscation), and the sidecar JAR fills the same gap with even less coupling to the bytecode.

6.2 Generality

The format design is independent of the inferrer that produces the specifications. Any source of JML clauses — the inferrer of Article 1, Daikon [4], hand-written contracts, contracts produced by a future LLM-assisted refinement step — can be embedded by the same mechanism. The format also generalises to languages other than Java that target the JVM (Kotlin, Scala) at the cost of those languages providing their own translators between source-level annotations and the bytecode `@JmlSpec` representation.

For non-JVM languages, the OpenAPI extension carries equivalent information at the service boundary. The bytecode side is JVM-specific; the service-boundary side is language-agnostic.

6.3 Limitations

The embedder is consumer-tolerant but the consumer has work to do. A reader that wants to act on the embedded clauses must (a) parse the canonical-form JML strings, (b) substitute parameter names, and (c) decide what to do with the result. The article ships the writer, the reader, and the canonicaliser, but does not ship a verifier or a runtime checker — those are downstream concerns that benefit from the work here without being implemented by it.

The 3% fidelity gap reported in Section 5 is an artefact of multi-release JAR keying, not a limit of the carrier. The follow-up release closes the gap. A more substantive limitation: roundtrip equality is on the canonical-form string, not on a normalised semantic theory, so equivalent specifications written in different forms will appear distinct on the wire. This is the deliberate separation-of-concerns chosen in Section 3; semantic normalisation is a separable layer.

7 Threats to Validity

Internal validity. The 100% roundtrip-fidelity figure depends on the writer covering every method shape that ASM emits. Section 5 discusses the original 3% gap that arose when the writer's lazy emission missed the abstract / interface case (only `visitEnd` fires for them). Adding `visitEnd` to the trigger set closed the gap on the libraries tested. The same risk pattern applies to method shapes specific to future JVM features — e.g., abstract record components, hidden classes (JEP 371) — which would need their own triggers if they exhibit different visit-call orderings. The mitigation is the same: a corpus-scale validation per JVM release.

The byte-overhead figure is measured under a deliberately bulky synthetic-spec workload (one annotation per parameter for every method). Real inferred specifications are typically smaller per method, so the reported 11.76%–13.49% range should be read as an upper bound for the embedder's contribution to JAR size. The 13.49% figure on Commons IO reflects its higher proportion of interfaces (each abstract method receives an embedded annotation just like a concrete method); a workload-aware embedder that elides annotations on abstract methods whose specs are pure pass-throughs would lower the figure further.

External validity. The two libraries reported are mature, well-structured, and ship through Maven Central. Generalisation to less-tidy code (older artefacts, vendored dependencies, hand-tuned ProGuard-stripped JARs) will need separate validation. The negative test confirms that the consumer-side contract (load classes without the annotation type) holds at JDK 21; older JDKs (8, 11, 17) and ahead-of-time compilation paths (GraalVM `native-image`) are deferred.

The OpenAPI extension is validated by construction (it is a JSON Schema valid against OpenAPI 3.x's extension rules). Empirical validation against a real microservice is part of the planned follow-up evaluation but does not appear in the present article.

Construct validity. "Roundtrip fidelity" is operationalised as set-equality on `(internalName, methodName, descriptor)` keys. Two methods with identical signatures but different bytecode bodies (for example, the same constructor compiled with different optimisation levels) appear identical to the harness. This is the right construct for the embedder's job — the embedder carries metadata, not bytecode — but a reader that needs to attach annotations to a specific compilation of a method would need additional disambiguation.

Conclusion validity. The empirical claims are point measurements on two libraries; no statistical significance test is meaningful. The claims are robust in the sense that the gap-causing patterns are identified and fixable, and the absolute throughput numbers are well above the thresholds CI/CD workflows require, but a wider corpus (the protocol's full ten libraries) is required before the claims generalise.

8 Related Work

Specification languages. JML [6, 7] provides the contract vocabulary this article transports. Eiffel's design-by-contract [8] originated the broader approach; Spec# [1] ported it to .NET. None of these languages address the distribution problem this article tackles — they assume the consumer has source.

Specification inference. Article 1 is the immediate predecessor. Daikon [4] infers likely invariants from execution traces; Houdini and its descendants infer candidate annotations and discharge them through a verifier. PreInfer [10] uses dynamic symbolic execution. Jdoctor [2] extracts exception preconditions from Javadoc. None of these systems address what happens to the inferred specifications after they are produced; the present article is the missing layer.

Annotation-based contracts. JSR-305 captured nullability annotations and was withdrawn before final standardisation; the surviving informal usage covers only nullability. The Checker Framework's annotation vocabulary is type-system-shaped and does not generalise to the relational and quantified clauses JML supports. Cofoja (Contracts for Java) emits runtime checks from `@Requires`/`@Ensures` annotations but does not survive without its annotation processor on the classpath; the present work is consumer-tolerant by design.

Bytecode metadata transport. ASM [5] is the standard library for class-file rewriting and underpins the embedder's implementation. ByteBuddy and Javassist provide higher-level APIs over similar primitives. None of these libraries prescribe a metadata format for specifications — they are mechanism, not policy. JEP 181 (nest-mates) and JEP 371 (hidden classes) extend what the JVM accepts as class-file content but do not change the annotation channel.

API description. OpenAPI 3.x is the standard for HTTP service descriptions; gRPC's `.proto` files cover RPC. Pact [9] captures consumer-driven contracts as example interactions; it is example-based rather than logical, so it is a transport for examples rather than for specifications, and the present article specifically rejects example-based encoding (Section 3). The OpenAPI extension proposed here is the smallest viable addition to an existing widely-deployed format.

Mutation testing and test generation. Article 1 cited the mutation-testing literature in detail; the present article inherits those citations through the bibliography. The relationship is: Article 1 measured what specifications do for LLM-generated tests, and Article 2 makes the specifications transportable to LLMs that do not have the source. Together they support the planned follow-up on service-boundary code (Article 3 and Article 4 in this thesis).

9 Conclusion

This article asked whether automatically inferred specifications can be embedded in compiled Java artefacts and REST API descriptions with sufficient fidelity that downstream consumers can use them as if they had the source. The answer is yes, with two specific constructions:

1. A repeatable runtime-retained Java annotation, `@JmlSpec`, written into class-file `RuntimeVisibleAnnotations` attributes by an ASM-based embedder. Roundtrip fidelity is 100.0% on three real-world libraries (Apache Commons Lang, Apache Commons IO, Guava — totalling 20,546 methods); bytecode-size overhead is 11.76%–13.49%; embedding throughput is 10,000–28,000 methods/second; reading throughput is 146,000–235,000 methods/second.

2. An OpenAPI 3.x extension family (`x-jml-requires`, `x-jml-ensures`, `x-jml-assignable`, `x-jml-signals`, `x-jml-invariant`) per-operation and per-schema, plus a sidecar JSON fallback for documents generated by frameworks. The extension is preserved by standard OpenAPI parsers without modification.

The negative test confirms that classes embedded with `@JmlSpec` load correctly in JVMs that have no awareness of the annotation type — the embedder is consumer-tolerant by design. Together with the inference engine of Article 1, the path from source code to consumer-accessible contract is complete: the inferrer produces specifications, the embedder ships them alongside compiled artefacts, and the OpenAPI extension carries them across service boundaries.

The follow-up work has three threads. First, validation across the remaining eight libraries identified in the protocol. Second, the integration with the LLM-based test generator of Article 1, which can now consume specifications directly from the JAR rather than requiring source. Third, the planned third article in this thesis, which extends the inference itself to compose specifications across method boundaries; the carrier built here is what makes that composition transportable to consumers.

The replication package, including the full source of the embedder/extractor pair, the OpenAPI extension JSON Schema, and the empirical validation harness, accompanies this article.

Data Accessibility

A complete replication package is available at `https://github.com/[anonymized]/jml-inferrer/jml-embedder`, containing the embedder/extractor source, the OpenAPI extension JSON Schema, raw measurement logs from the empirical validation, and reproducibility scripts.

Acknowledgments

This research was supported by the University of Queensland Cyber initiative.

Conflict of Interest

The authors declare no conflict of interest.

Author Contributions

Brendan Edmonds: Conceptualization, Methodology, Software, Validation, Investigation, Data Curation, Writing - Original Draft, Visualization. **Mark Utting**: Conceptualization, Methodology, Writing - Review & Editing, Supervision.

ORCID

Brendan Edmonds https://orcid.org/0000-0000-0000-0000
Mark Utting https://orcid.org/0000-0000-0000-0000

References

[1] Mike Barnett and K. Rustan M. Leino. Weakest-precondition of unstructured programs. In *Workshop on Program Analysis for Software Tools and Engineering (PASTE)*, pages 82–87, 2005. doi: 10.1145/1108792.1108813.

[2] Arianna Blasi, Alberto Goffi, Konstantin Kuznetsov, Alessandra Gorla, Michael D. Ernst, Mauro Pezzè, and Sergio Delgado Castellanos. Translating code comments to procedure specifications. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA '18, pages 242–253. ACM, July 2018. doi: 10.1145/3213846.3213872. URL http://dx.doi.org/10.1145/3213846.3213872.

[3] David R. Cok. OpenJML: Software verification for Java 7 using JML, OpenJDK, and Eclipse. In *1st Workshop on Formal Integrated Development Environment (F-IDE)*, volume 149 of *EPTCS*, pages 79–92, 2014. doi: 10.4204/EPTCS.149.8.

[4] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69(1–3):35–45, 2007. doi: 10.1016/j.scico.2007.01.015.

[5] INRIA / OW2 Consortium. ASM Java bytecode manipulation framework, 2024. URL https://asm.ow2.io/.

[6] Gary T. Leavens, Albert L. Baker, and Clyde Ruby. Preliminary design of JML: A behavioral interface specification language for Java. *ACM SIGSOFT Software Engineering Notes*, 31(3):1–38, 2006. doi: 10.1145/1127878.1127884.

[7] Gary T. Leavens et al. JML reference manual. http://www.jmlspecs.org, 2013. Accessed: 2025-01-15.

[8] Bertrand Meyer. Applying "design by contract". *Computer*, 25(10):40–51, 1992. doi: 10.1109/2.161279.

[9] Pact Foundation. Pact: contract testing for HTTP and message integrations, 2024. URL https://pact.io/.

[10] Xinyu Wang, Isil Dillig, and Ruzica Piskac. PreInfer: Inferring preconditions via symbolic execution. In *ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 779–793, 2017. doi: 10.1145/3133876.